34. Concurrent Algorithm Drills

34. Concurrent Algorithm Drills

Phase: Phase 5 — Concurrency
Estimated time: 2 days
Slug: `34-concurrent-algorithms`

Learning objectives

- Parallelize map-reduce style workloads
- Compute safely with bounded concurrency
- Compare sequential vs parallel benchmarks
- Preserve determinism in tests

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Concurrent algorithms combine correct partitioning with sync. Parallel sum/map patterns prepare you for real data pipelines.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Split input into shards; process independently; merge.
- Avoid false sharing conceptually; keep shards separate.
- WaitGroup + results channel is a common merge pattern.
- Correctness first, then speedup measurements.
- Not all problems parallelize — Amdahl's law.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-concurrent && cd /tmp/gw-concurrent
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import (
    "fmt"
    "sync"
)

func ParSum(xs []int, workers int) int {
    if workers < 1 {
        workers = 1
    }
    ch := make(chan int, workers)
    var wg sync.WaitGroup
    n := len(xs)
    size := (n + workers - 1) / workers
    for w := 0; w < workers; w++ {
        lo := w * size
        if lo >= n {
            break
        }
        hi := lo + size
        if hi > n {
            hi = n
        }
        wg.Add(1)
        go func(part []int) {
            defer wg.Done()
            s := 0
            for _, v := range part {
                s += v
            }
            ch <- s
        }(xs[lo:hi])
    }
    go func() { wg.Wait(); close(ch) }()
    total := 0
    for s := range ch {
        total += s
    }
    return total
}

func main() { fmt.Println(ParSum([]int{1, 2, 3, 4}, 2)) }
```

Run it:

```
go run .
```

Expected output:

```
10
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Parallel sum with 1 vs 4 workers; check equal results.
2. Benchmark both.
3. Break merge by double-closing a channel — then fix.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Drill themes

- Parallel sum / histogram with shard-per-worker then merge
- Concurrent crawler with URL graph + visited set (mutex or owning map goroutine)
- Pipeline: read → parse → enrich → write

Rules

- Bound parallelism to GOMAXPROCS or a chosen limit
- No shared map writes without sync
- Tests must be deterministic (sort outputs)

Measure: sometimes parallel is slower due to coordination overhead — learn when.

Failure modes to watch while coding

- Racing on shared maps
- More goroutines than work items blindly
- Non-deterministic test flakes ignored

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- multiprocessing vs goroutines — different memory models.
- Shared-memory parallel needs locks in both.
- Chunking strategy matters more than language.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

For tiny inputs, parallel overhead wins — gate on size.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Concurrent Algorithm Drills” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Racing on shared maps
- More goroutines than work items blindly
- Non-deterministic test flakes ignored

Exercises

Work in `exercises/34-concurrent-algorithms/`. Read that folder’s `README.md` before you code.

What you will build

Implement parallel aggregation matching sequential results.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/34-concurrent-algorithms/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. When parallelization fails to help?
2. How to merge partial histograms?
3. Why sort in tests?

Further reading

- https://pkg.go.dev/sync